Optimizing Bitmap-Based Multi-Version Management in Columnar HTAP Databases (Technical Report)

Qianlu Wu[1], Dawei Liu[1], Guoneng Li[1], Ling Xu[2],
Bo Gao[2], Haixin Wang[3], Zhijun Wang[2], Wei Wang[2], Bolong Zheng[1]
[1]*Huazhong University of Science and Technology, China*
[2]*Shuyi Technology Co., Ltd., China* [3]*Wuhan University of Technology, China*

***Abstract*—The rise of real-time digital applications demands efficient Hybrid Transactional/Analytical Processing (HTAP) systems that support high-frequency transactional updates as well as complex analytical query processing. While traditional separate OLTP/OLAP architectures incur costly ETL processes, modern HTAP designs favor in-memory columnar storage with Multi-Version Concurrency Control (MVCC) for concurrency. However, in-memory MVCC suffers from significant memory overhead caused by accumulation of redundant versions, as long-running analytical queries that can stall garbage collection and prevent timely reclamation, leading to degraded performance. In this paper, we present HierDiff, a novel bitmap-based multiversion management framework that addresses these limitations. HierDiff achieves efficient memory utilization and competitive system performance across workloads via the following mechanism: (i) a hierarchical grouped differential encoding scheme with a two-level chain structure to compress redundant bitmaps; (ii) an adaptive garbage collection mechanism with inter-group merging for timely reclamation; and (iii) an immediate commit protocol via a Delete Log to minimize thread conflicts. As a pluggable module, HierDiff integrates seamlessly with bitmap-based HTAP systems such as HexaDB. Extensive experiments demonstrate that HierDiff reduces multiversion memory overhead by up to $12.3\times$, while achieves comparable throughput and latency with HexaDB.**

***Index Terms*—HTAP, MVCC, Columnar storage**

I. Introduction

In the era of digital commerce and real-time decision-making, modern applications increasingly demand seamless integration of transactional and analytical workloads to handle vast, dynamic datasets efficiently. For example, e-commerce platforms like Amazon or Taobao need to support high-frequency point updates, such as processing millions of orders per second, while concurrently performing complex analytical queries in milliseconds. Traditional architectures address this issue by maintaining separate OLTP and OLAP databases, which are synchronized via periodic Extract, Transform, Load (ETL) processes. However, the inherent latency of ETL pipelines results in data staleness that can lead to revenue losses or delayed insights in business decisions. To tackle this problem, the industry has shifted towards Hybrid Transactional/Analytical Processing (HTAP) systems, which are designed to unify these disparate workloads on a single data store. With exploding data volumes, the need for efficient HTAP systems has become essential.

To enable the dual workloads, HTAP systems need to efficiently serve two access patterns: Analytical Processing (AP) involves long-running queries that scan and aggregate large datasets [1], while Transactional Processing (TP) consists of short, latency-sensitive operations requiring strong consistency [2]. A common architecture is to integrate row-oriented storage for fast point updates with column-oriented storage optimized for scanning, compression, and vectorized execution in analytics [3], [4]. Yet, achieving high concurrency in such a hybrid environment requires robust mechanisms to manage simultaneous reads and writes without conflicts.

Multi-Version Concurrency Control (MVCC) is the cornerstone for achieving transactional isolation and high concurrency in HTAP systems [5]–[7]. By maintaining multiple versions of each logical record, MVCC enables read-only analytical queries and write-intensive transactions to execute concurrently without costly locking. Despite these benefits, MVCC suffers from substantial storage overhead from version accumulation and the associated metadata management burdens. Traditional database systems mitigate this issue by offloading obsolete versions to disk [3], [8]. However, this strategy brings critical performance degradation, as accessing or storing these versions incurs expensive I/O operations.

Another approach involves caching MVCC versions entirely in memory to preserve transactional consistency and minimize I/O costs. Nevertheless, this approach encounters significant challenges in HTAP environments, where mixed workloads amplify version management complexities. First, current bitmap-based solutions overlook the high redundancy among consecutive bitmap versions. This limitation is particularly pronounced in visibility bitmaps, a common structure used in columnar stores for tracking record visibility. To avoid the overhead of frequent compression and decompression, most designs choose to store each bitmap version in its entirety [9], [10]. Although this strategy prioritizes low-latency transactional operations, it leads to significant waste of memory resources. Second, existing garbage collection (GC) mechanisms struggle to reclaim obsolete versions promptly [9], [11]. Most systems adopt coarse-grained policies. The classic *oldest*

active transaction policy relies on a single global reclamation boundary determined by the oldest active snapshot. Although more scalable alternatives such as interval-based GC [12] and epoch-based schemes [13] have been proposed, they remain inherently conservative. Any interval or epoch that partially overlaps with active snapshots must still be preserved in full. Consequently, many invisible intermediate versions cannot be promptly excised, leading to substantial accumulation of redundant bitmap versions and noticeably increased memory overhead under long-running analytical workloads.

These limitations highlight a critical research gap: *existing MVCC designs fail to exploit the substantial inter-version similarity for efficient compression and timely reclamation under long-running queries*. As a result, current systems are forced into a fundamental trade-off between memory efficiency and access latency. Our analysis reveals that adjacent bitmap versions often differ only slightly (e.g., less than 0.02%). This key observation motivates the use of lightweight differential encoding to reduce memory overhead dramatically without compromising runtime efficiency. Therefore, this paper rethinks in-memory columnar MVCC to design a storage architecture that resolves this trade-off.

Building on these insights, we propose HierDiff, a novel framework for bitmap-driven multiversion management that enhances MVCC efficiency in HTAP environments. The core idea is to utilize the high similarity among adjacent bitmap versions to design efficient mechanism that saves memory consumption. Specifically, HierDiff incorporates a hierarchical grouped differential encoding scheme along with a two-level version chain structure to achieve high compression ratios. To address the memory-efficiency limitations caused by coarse-grained garbage collection, it employs an adaptive garbage collection mechanism coupled with inter-group merging for timely reclamation of obsolete versions. Finally, to maintain high system efficiency, HierDiff introduces an immediate commit protocol that decouples version updates from the transaction path to minimize thread conflicts and latency. As a pluggable module, HierDiff can be easily integrated with various bitmap-based HTAP databases to satisfy the diverse client requirements.

To summarize, the contributions of this paper are as follows:

- To the best of our knowledge, this is the first work investigating efficient memory management of bitmap data structures in MVCC under HTAP workloads.
- We identify the critical pain points of existing in-memory MVCC designs under HTAP workloads, including massive memory waste from storing redundant, uncompressed version data and limited memory efficiency caused by coarse-grained garbage collection strategies.
- We propose a novel bitmap-based MVCC framework named HierDiff, which includes a hierarchical grouped differential encoding scheme to save memory consumption, an adaptive garbage collection mechanism to reclaim obsolete versions timely, and an immediate commit protocol to minimize thread conflicts.
- We integrate HierDiff into the existing HexaDB HTAP database system and conduct extensive experimental evaluations. The experimental results demonstrate that our proposed solution reduces the memory requirements by up to $12.3\times$, while achieves comparable throughput and latency with HexaDB.

II. Background and Preliminaries

A. Row and Column Stores in HTAP Databases

Modern data management workloads broadly fall into Online Transaction Processing (OLTP) and Online Analytical Processing (OLAP). OLTP involves frequent, low-latency operations such as inserts, updates, and deletes, while OLAP entails complex queries that scan and aggregate large datasets for insights. These demands have led to two storage paradigms: *row stores* and *column stores*. Row stores, such as PostgreSQL [14] and MySQL [15], organize data tuple by records. This layout is ideal for OLTP, as all attributes of a single record are co-located to enable efficient point accesses and updates. However, they underperform in OLAP, as analytical queries must read entire rows even when only a few columns are needed. In contrast, column stores excel at OLAP by storing all values of a single attribute contiguously, which maximizes data compression and enables high-speed vectorized scans [16], [17]. Nevertheless, column stores incur higher overhead for frequent updates, as attributes are dispersed across blocks.

Hybrid Transactional/Analytical Processing (HTAP) databases bridge this gap by integrating both formats in a single system to support mixed workloads efficiently [11], [18]. A common architecture is the Primary Column Store with a Delta Store [19]. In this model, the majority of data resides in a read-optimized columnar format for fast analytics. Recent updates and new data are staged in a write-optimized delta store, typically implemented as a row store or an in-memory buffer. For example, HexaDB [20], a modern HTAP-oriented database system optimized for MVCC memory management, adopts this architecture. HexaDB allocates about 20% of data to a row store for high-frequency updates and 80% to a column store for analytics. At runtime, hot data is first stored in an in-memory, row-based cache to enable efficient updates. As the data becomes less frequently accessed, it is gradually migrated to a column-based cache optimized for analytical processing. The cached data is periodically persisted to disk for durability.

B. Multi-Version Concurrency Control

Multi-Version Concurrency Control (MVCC) is the dominant concurrency control protocol in modern databases. It is designed to maximize parallelism by allowing reads and writes to proceed without blocking [5]. Instead of using locks, MVCC creates a new version of a data item upon modification, and each transaction operates on a consistent snapshot of the database defined by its transaction ID and the current isolation level [21], [22].

In row stores, MVCC typically employs in-record version chains, where each record directly links to its predecessor [3], [23], [24]. Column stores, however, decouple version metadata

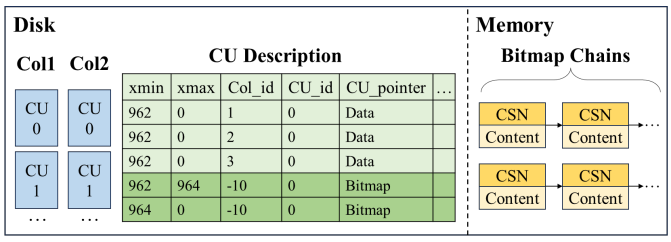


Fig. 1: MVCC structure in HexaDB

from physical data layout, using bitmaps or visibility vectors per column for tuple tracking [25]. In some column-store systems, MVCC metadata are persisted on disk for durability, while being loaded into memory during query execution to enable efficient vectorized scans. Other systems further offload version metadata to disk or cold regions to reduce memory usage [9], [16], [17], [26]. However, the accumulation of obsolete versions can cause version bloat [27], where outdated data occupy memory storage and degrade query performance. To clean up the outdated data, garbage collection (GC) threads periodically reclaim them. Its effectiveness directly affects both memory utilization and transaction latency, making it a crucial component of MVCC systems.

C. MVCC Design in HexaDB

Our work builds upon and enhances the in-memory MVCC mechanism in HexaDB. Below, we present its core design elements to provide context for our optimizations.

Data Layout and Versioning. HexaDB employs a primary column store with a delta store, which serves as an in-memory row cache for recently accessed hot data. In its column store, each column is partitioned into Compression Units (CUs). Versioning is managed not at the tuple level, but at the CU level using delete bitmaps. Each CU is compressed and recorded in a `CU Description Table`. As illustrated in Fig. 1, each entry of the table contains: (i) the creating and invalidating transaction identifiers (`xmin` and `xmax`), (ii) the logical identifiers `Col_id` and `CU_id` that uniquely locate the CU within the column, and (iii) a pointer `CU_pointer` whose interpretation is determined by the sign of `Col_id`: a positive value points to physical CU data, while a negative value (fixed at -10) references the corresponding version bitmap. During transaction processing, the version bitmaps evolve according to the `xmin`/`xmax` visibility rules, with new versions appended and persisted as updates propagate. In HexaDB, the `CU Description Table` persistently stores CU bitmap states and incorporates changes as updates propagate.

In-Memory Version Chains. To optimize for low-latency access, HexaDB materializes all version bitmaps in main memory to avoid disk I/O penalities. For each CU, these bitmaps form a version chain, implemented as a linked list. When a transaction modifies a CU, it creates a new bitmap version and prepends it to the head of the chain. Each version is tagged with a monotonically increasing Commit Sequence Number (`CSN`). A transaction determines visible versions by traversing the chain and comparing its own snapshot `CSN` against the `CSNs` of the bitmap versions. Only versions with $\texttt{CSN} \leq \textit{snapshot}\ \texttt{CSN}$ are visible. Therefore, the in-memory version chain accelerates runtime visibility evaluation, while the `CU Description Table` provides the persistent storage foundation for bitmap state.

Garbage Collection. The GC in HexaDB follows a standard *oldest active transaction* policy. The system tracks the smallest CSN among all active transactions as `MinActiveXID`. A background `VACUUM` process is periodically triggered to reclaim obsolete versions whose CSNs are smaller than `MinActiveXID`. While effective in simple scenarios, this approach directly exposes the system to severe version bloat whenever a long-running analytical query is present. This inefficient memory management is the primary bottleneck we aim to solve.

III. Problem Analysis and Design Overview

A. Key Challenges

Our work addresses two fundamental bottlenecks in existing in-memory MVCC designs under HTAP workloads. These challenges highlight a critical trade-off between access latency and memory efficiency that current systems fail to resolve.

Massive Redundancy in Versioned Bitmaps. The first bottleneck lies in how versions are physically stored. To avoid the computational overhead of on-the-fly compression and decompression, existing systems typically store each version in its entirety. However, our analysis reveals that this approach results in massive data redundancy, particularly for structures like visibility bitmaps. Under typical HTAP workloads, individual transactions typically modify only a small fraction of records within a data block. Consequently, the visibility bitmaps of adjacent versions have extremely high similarity, with only a few bits changing between them.

To quantify the redundancy, we conduct a preliminary experiment on the TPC-C benchmark. By calculating the Hamming distance between consecutive bitmap versions across various columnar blocks, we observe an average version similarity exceeding 99.98%. Storing each nearly identical version as a full copy is therefore significantly wasteful. This observation highlights a clear opportunity: lightweight differential encoding could drastically reduce memory consumption without a significant performance penalty, thereby resolving the core trade-off between memory efficiency and access latency.

Coarse-Grained GC Limiting Memory Efficiency. In HTAP environments, concurrent short-lived transactional updates and long-running analytical queries challenge MVCC garbage collection. Existing GC strategies for version chains remain coarse-grained. The classic *oldest active transaction* policy establishes a single global reclamation boundary based on the oldest active snapshot. Under this policy, intermediate versions newer than the oldest active snapshot may still be retained even when they are no longer visible to any active transaction. This "overly conservative" design can cause uncontrolled version bloat: a rapid, unbounded growth in memory consumption that degrades system performance and ultimately leads to out-of-memory failures.

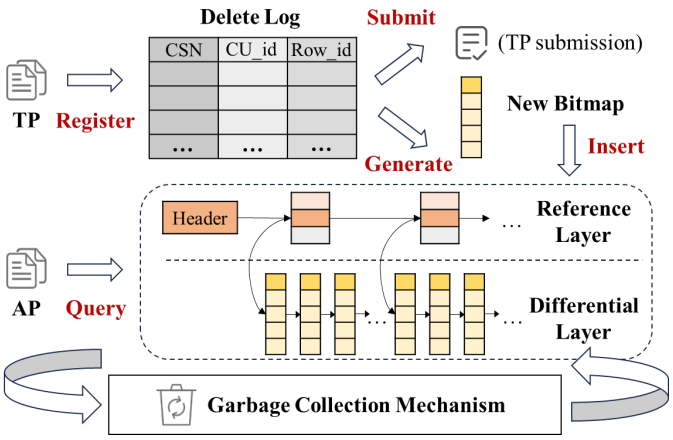


Fig. 2: Workflow of HierDiff

Modern refinements such as interval-based GC improve scalability by grouping snapshot CSNs into fixed-size ranges and reclaiming at interval granularity. Nevertheless, any interval that partially overlaps with active snapshots must still be preserved conservatively, leaving redundant intermediate versions un-reclaimed. Epoch-based schemes, on the other hand, primarily target deferred safe memory reclamation after nodes have already been unlinked from shared structures. They do not address the core problem of precise logical pruning of bitmap versions according to MVCC visibility rules.

As a result, all these approaches cannot promptly identify and excise invisible intermediate versions under long-running analytical workloads. Consequently, redundant bitmap versions accumulate over time, leading to substantial memory overhead, longer version chains, and noticeably reduced memory efficiency in HTAP scenarios.

B. Design Goals

Guided by our analysis of MVCC challenges in HTAP systems, we outline the key objectives for an optimized in-memory version management strategy. To this end, we establish the following design goals for our system:

- **Optimized memory utilization**: The system should minimize per-version storage overhead by leveraging inter-version similarities without compromising data integrity and consistency.
- **Competitive performance across workloads**: The versioning mechanism should support both high-throughput transactional operations and low-latency analytical queries. The overhead of version reconstruction should be minimal to ensure that neither workload type is penalized.

C. HierDiff Overview

To fulfill our design goals, we propose HierDiff, a bitmap-based multiversion management framework that optimizes MVCC for HTAP systems. HierDiff comprises three synergistic components tailored for optimized memory utilization and balanced performance across workloads.

For efficient memory utilization, we design a hierarchical grouped differential encoding scheme with a two-level chain structure that leverages the high similarity among adjacent

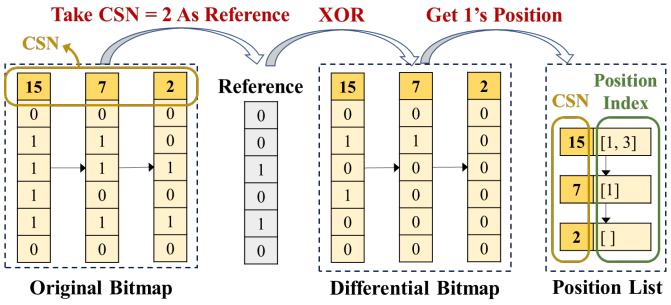


Fig. 3: Basic XOR-based differential compression

bitmap versions. This strategy achieves substantial compression without excessive reconstruction overhead. It not only reduces storage demands but also accelerates visibility checks and query execution by limiting the data processed during traversals. To further reduce memory consumption, we propose an adaptive garbage collection mechanism coupled with group merging, which promptly reclaims obsolete versions and dynamically consolidates sparse groups. This approach maintains optimal compression ratios while shortening version chain lengths to enhance traversal speed and overall performance. For balanced performance across workloads, we devise an immediate commit protocol via a `Delete Log`, decoupling version updates from transactional paths to minimize thread conflicts and latency. This ensures seamless handling of both high-throughput TP operations and long-running AP queries.

Workflow. Fig. 2 illustrates the workflow of HierDiff. For insert operations, along with the creation of a new Compression Unit (CU), a two-level chain with hierarchical grouped differential encoding scheme is initialized. For deletes and updates, the immediate commit protocol is executed to register operations into a `Delete Log`. Then background threads scan the `Delete Log` top-down, generate new bitmaps, and insert them into the two-level chain. For AP operations, queries directly scan the two-level chain and return the result. Apart from HTAP transactions, the garbage collection mechanism is adopted by the background thread, removing obsolete versions in the two-level chain.

IV. HierDiff Design

A. Hierarchical Grouped Differential Encoding

Traditional single-level bitmap version chains offer straightforward insertion (for database inserts, deletes, and updates) and querying. However, as analyzed in Sec. III-A, this design incurs substantial redundancy by storing full bitmaps for each version. Observing the sparsity of updates between adjacent versions, our approach exploits this high similarity through differential encoding. Nonetheless, a naive single-reference approach, where all subsequent versions are encoded as differences from the very first version, rapidly loses its effectiveness as changes accumulate over time.

Fig. 3 illustrates the bitmap chain structure in HexaDB, where compressed versions for a CU are organized into a linear chain. Each version stores its `CSN` and a bitmap denoting the deletion status of each record position within the CU. A basic

differential strategy selects one version as the `Reference` version, while subsequent versions are encoded as XOR-based differential bitmaps relative to that `Reference`. These XOR-compressed bitmaps typically contain only a small number of "1" bits, representing modified positions. Then, it records the positions of 1 using a position list. However, as TP transactions introduce increasingly dispersed updates, the number of versions increase. As a result, the compression ratio declines if all differential bitmaps are derived from a single `Reference`.

To address this issue, we propose a hierarchical grouped differential encoding scheme featuring a two-level version chain structure. The core concept involves partitioning versions into groups and compressing within each by leveraging intra-group similarity. As shown in Fig. 4 , we partition the linear version chain into smaller, manageable groups. Each group contains one full `Reference` version and several differential versions. Only the `Reference` version is stored as a full bitmap. Subsequent versions are stored as compact differentials relative to that Reference. This two-level version chain structure allows to achieve high compression ratios by keeping differentials sparse, while ensuring fast version reconstruction for both analytical and transactional queries. Fig. 4 further details the `Reference` version, each maintaining: (1) a `CSN` range corresponding to its attached differentials, defining the subgroup's version interval; (2) a pointer to the first differential bitmap in the second-level chain; and (3) the complete bitmap of the `Reference` version itself.

Compared with conventional bitmap compression methods such as WAH [28] and Roaring Bitmap [29], the grouped differential compression of HierDiff is particularly well-suited for columnar MVCC workloads. It utilizes small Hamming distances between consecutive versions to yield superior compression ratios. As target version reconstruction requires only a single XOR operation, it effectively balances compression/decompression efficiency and query latency.

Optimal Group Size Analysis. The effectiveness of this scheme relies on choosing an optimal group size, denoted by m. A small m leads to too many `Reference` versions, which increases memory consumption. A large m causes differentials to become dense, reducing compression efficiency. To formalize this trade-off, we model the version evolution and derive the optimal group size m^*that minimizes total memory cost.

Let p denote the probability that a bit flips from 0 to 1 between consecutive versions, n denotes the total number of bitmap versions in a two-level chain, m be the group size (i.e., maximum number of differential bitmaps per `Reference`), and L be the number of bits in each bitmap.

Definition 1 (Bit-Flip Model)**.** For each pair of consecutive versions, every bit that is currently 0 independently flips to 1 with probability p, following a binomial distribution over the L bits. Once a bit becomes 1, it remains unchanged in all subsequent versions.

Definition 2 (Expected Bit Accumulation Across Versions)**.** Given the bit-flip model above, the expected number of 1 bits

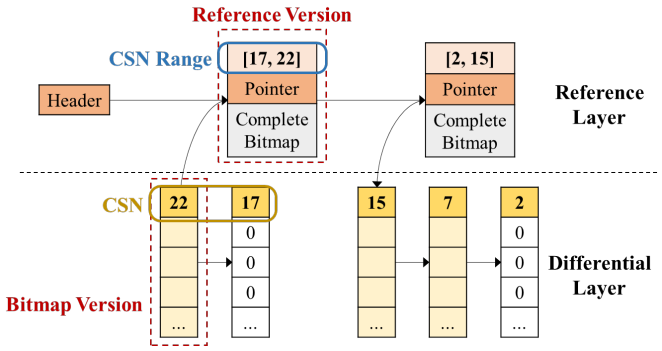


Fig. 4: Structure of grouped differential compression

in the i-th version is

$$\mathrm{E}_{bits}(i) = L\big(1-(1-p)^i\big).$$

For the z-th group, the first version is the $(z-1)m+1$-th version in the whole two-level chain, the j-th version is the $(z-1)m+j$-th version, and the last version is the zm-th version. Since the first version in each group is chosen as the `Reference`, using the expectation derived in Def. 2, the number of 1 bits newly introduced in the j-th version of the z-th group is

$$\mathrm{E}_{\Delta}(z,j) = \mathrm{E}_{bits}\big((z-1)m+j\big) - \mathrm{E}_{bits}\big((z-1)m+1\big).$$

To determine the optimal group size that minimizes total memory cost, we formulate the expected space consumption for a whole two-level chain as

$$f(m) = 16\sum_{z=1}^{\frac{n}{m}}\left(\sum_{i=(z-1)m+1}^{zm}\mathrm{E}_{\Delta}(z,i)\right) + \frac{nL}{m},$$

where the coefficient 16 reflects the storage cost of the differential-compressed bitmap: each newly introduced bit with value 1 is represented by a 16-bit entry in the position list. The inner summation accounts for the expected number of newly introduced 1 bits across all differential versions within the z-th group, while the outer summation aggregates this cost over all groups in the two-level version chain. The final term $\frac{nL}{m}$ represents the space overhead of storing the `Reference` versions, with one full L-bit bitmap per group.

Let $t = 1-p$. By simplifying the above summations, we obtain the closed-form expression:

$$f(m) = 16Lt(1-t^n)\cdot\frac{m-\frac{1-t^m}{1-t}}{1-t^m} + \frac{nL}{m}.$$

To find the optimal group size m^*, we take the derivative with respect to m:

$$f'(m) = 16Lt(1-t^n)\cdot\frac{1-t^m+mt^m\ln(t)}{(1-t^m)^2} - \frac{nL}{m^2}.$$

Setting $f'(m)=0$ yields the condition for optimal m:

$$16t(1-t^n)\big[1-t^m+mt^m\ln(t)\big] = \frac{n(1-t^m)^2}{m^2}.$$

This transcendental equation can be solved numerically to obtain the minimal expected memory cost. For small p (i.e., when $pm \ll 1$), we approximate $(1-p)^m \approx 1-pm$ and

$\ln(1-p) \approx -p$, leading to

$$m^* \approx \sqrt{\frac{n}{8t(1-t^n)}}.$$

This analytical result indicates that the optimal group size grows sublinearly with the total number of bitmap versions.

B. Garbage Collection and Group Merging

Differential encoding reduces per-version storage but cannot prevent obsolete versions from accumulating. As discussed in Sec. III-A, the *oldest active transaction* policy may retain obsolete intermediate versions newer than the oldest active snapshot. Interval-based GC [12] refines reclamation granularity but can still retain redundant versions within partially active intervals. Epoch-based schemes [13] address safe memory reclamation after unlinking, rather than snapshot-aware logical pruning. These limitations motivate finer-grained identification of obsolete bitmap versions using active snapshots.

HierDiff employs a visibility-oriented reclamation strategy. Instead of retaining versions according to a single global frontier or coarse interval boundaries, HierDiff periodically scans the active transaction table and collects the snapshot `CSN`s of all running transactions. For each active transaction, the *visible* bitmap version is defined as the first version encountered when traversing the chain from newest to oldest whose `CSN` does not exceed that transaction's snapshot `CSN`. Equivalently, it is the unique bitmap version that would be returned by snapshot lookup for that transaction. These snapshot-visible versions must be preserved because they may still be accessed by active transactions. By contrast, any intermediate version that is not visible to any active transaction is no longer needed for visibility and can be safely regarded as obsolete. Such obsolete versions are reclaimed asynchronously by background workers to minimize interference with foreground transaction execution. Thus, HierDiff can reclaim obsolete middle versions more precisely than both the *oldest active transaction* policy and interval-based GC, while avoiding the mismatch between epoch-based reclamation and snapshot-aware version pruning.

Fig. 5 illustrates an example of HierDiff's garbage collection. Suppose that the snapshot lookups of the three active transactions `T1`, `T2`, and `T3` return bitmap versions with `CSN`s 2, 7, and 22, respectively. In other words, when each of these transactions traverses the version chain using its own snapshot `CSN`, it stops at version 2, 7, or 22 because that is the first version whose `CSN` does not exceed its snapshot boundary. Assume further that no other bitmap version in the two-level chain is visible to any active transaction in the active transaction table. Then, versions 2, 7, and 22 must be retained, whereas all remaining intermediate versions are unreachable by active snapshots and can therefore be regarded as obsolete. Consequently, the background GC thread can safely scan the chain and reclaim all these unreferenced versions.

After versions reclamation, some groups may contain only a few remaining differential versions, which diminishes the overall compression efficiency. To maintain a compact structure and fully leverage the differential encoding scheme, HierDiff

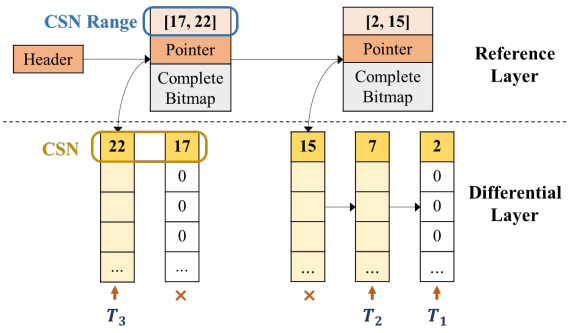


Fig. 5: Example: garbage collection

incorporates an *group merging* mechanism that dynamically combine groups containing few versions with other groups.

Once the number of differential versions within a group drops below a predefined threshold $\mathcal{T}$, HierDiff triggers a merge with its adjacent group. Specifically, when merging group A into group B, HierDiff first performs a bitwise XOR between the two `Reference` bitmaps to compute their differential bits, then flips the corresponding bits of each version in A accordingly. Subsequently, it changes the pointer of B's `Reference` version and the pointer of A's last bitmap version, which indicates joining the differential layer chain in A to that in B. Finally, the `Reference` version of the emptied group is removed. This adaptive merging process effectively reduces chain fragmentation and eliminates redundant nodes, preserving both spatial efficiency and query performance. As illustrated in Fig. 6, the version with $CSN = 22$ is merged into the second group, and the first `Reference` version is discarded after the merge is completed. Note that group merging does not compromise overall compression efficiency. As quantified below, merging is triggered only when the number of remaining versions in a group falls below the threshold $\mathcal{T}$. This threshold is derived exactly from the condition that the extra second-layer cost introduced by re-encoding these versions is smaller than the space saved by removing one `Reference` version.

To minimize memory usage, the merging threshold $\mathcal{T}$ must ensure that merging two adjacent groups reduces the overall space. When the $(z+1)$-th group reaches the threshold, it is merged into the zth group. Since a bit keeps unchanged once it becomes 1, the difference between the two `Reference` versions can be expressed using the formula in Def. 2:

$$\mathrm{E}_{bits}(zm+1) - \mathrm{E}_{bits}\big((z-1)m+1\big) = Lt^{zm}(t^{1-m}-t).$$

Let the remaining number of versions in the $(z+1)$-th group be v. Merging then introduces an extra second-layer cost of

$$16 \cdot v \cdot Lt^{zm}(t^{1-m}-t).$$

Merging is beneficial when this increase is smaller than the space saved by removing one `Reference` version (i.e., L bits). Hence, the remaining number of versions v in the $(z+1)$-th group must satisfy

$$v < \frac{1}{16 \cdot t^{zm}(t^{1-m}-t)}.$$

We thus define the right-hand side of the inequality as the

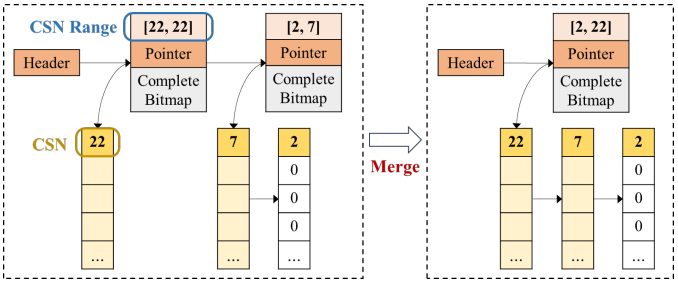


Fig. 6: Example: group merging

merging threshold $\mathcal{T}$.

Through this periodic garbage collection and adaptive merging mechanism, HierDiff maintains stable memory usage while preserving MVCC integrity, query and update efficiency.

C. Immediate Commit Protocol

The immediate commit protocol targets delete operations and append-only updates, primarily to mitigate thread conflicts caused by concurrent bitmap version insertions in the two-level chain. For insert operations, a new CU is initialized along with its corresponding two-level version chain. An initial bitmap version with all bits set to 0 is generated and linked to the head of the version chain. Subsequently, when a thread performs delete operation or append-only update, a new bitmap version needs to be inserted into the same version chain. However, concurrent chain insertions can cause significant contention, delaying transaction commits.

To address this issue, HierDiff introduces a lightweight `Delete Log` that records only the intent of each modification and decouples bitmap materialization from the cirticil execution path. During foreground transaction execution, each delete or update simply appends the affected `Cu_id` and `Row_id` to the `Delete Log`. When the transaction reaches the commit phase and obtains its commit `CSN`, HierDiff inserts an empty *placeholder* node into the version chain at the correct `CSN` position. This placeholder reserves the logical position of the new version before the transaction returns, ensuring that the commit latency remains minimal while preserving strict CSN monotonicity.

A dedicated background thread asynchronously processes the `Delete Log` and completes the remaining work for these placeholders. It scans log entries in ascending `CSN` order and groups them by `Cu_id` and `CSN`. For each group, it constructs the corresponding bitmap update by setting the bits of the recorded `Row_ids` to 1, indicating logical deletions of the affected rows, and uses the generated bitmap to fill the corresponding placeholder. Once the required predecessor version becomes visible, the new version is integrated into the chain and marked available. This procedure, named *Placeholder–Filling–Visibility*, is described in detail in Sec. V-A.

By decoupling expensive bitmap generation and integration from the transaction execution path, the immediate commit protocol significantly reduces transaction latency without compromising MVCC correctness. Queries that encounter an unfilled placeholder simply wait until the Filling and Visibility stages complete, thereby guaranteeing snapshot isolation semantics: once a transaction commits, its effects become visible to all subsequent snapshots in `CSN` order.

Regarding durability, HierDiff extends HexaDB's persistence path by introducing a durable `Delete Log`. Instead of forcing each fine-grained bitmap change to be immediately reflected on the foreground execution path, HierDiff first records the delta information of committed updates in the `Delete Log`. These deltas describe the differences with respect to the bitmap state maintained in the `CU Description Table`. Both the `CU Description Table` and the `Delete Log` are persisted on storage and protected by WAL, whereas the version chain and its placeholder nodes are maintained only as in-memory acceleration structures for fast bitmap access. After a crash or restart, the correct bitmap state can be reconstructed from the persistent bitmap information in the `CU Description Table` together with the durable delta records in the `Delete Log`. In this way, HierDiff removes expensive bitmap materialization from the foreground transaction path without sacrificing crash safety or durability.

V. System Design

A. Concurrency Control

In multi-threaded environments, committed delete intents recorded in the `Delete Log` may be materialized by background workers out of `CSN` order. If new bitmap versions are linked into the chain only after full bitmap construction, this could violate the strict monotonic `CSN` ordering required by MVCC visibility. Specifically, a version with a larger `CSN` may finish materialization earlier than those with a smaller `CSN`. A naive design could therefore allow a transaction to observe a newer committed state while missing an older one. To guarantee correctness and preserve a consistent version sequence, HierDiff adopts a three-stage protocol, referred to as the *Placeholder–Filling–Visibility* mechanism.

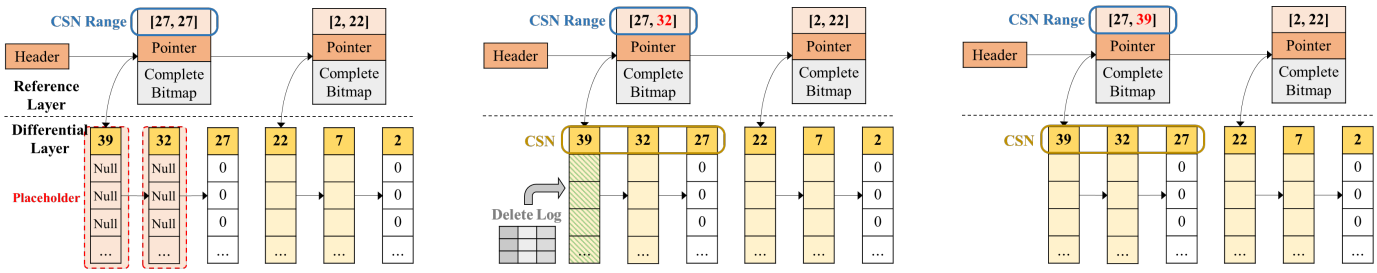
Placeholder. On the commit path, after a transaction obtains its commit `CSN`, HierDiff inserts an empty placeholder node into the proper position of the version chain and marks it unavailable. This step reserves the logical order of the committed version before the transaction returns.

Filling. A background worker later scans the `Delete Log`, collects the corresponding row-level update intents, and constructs the differential bitmap for the placeholder. This phase materializes the bitmap payload asynchronously without making the version visible.

Visibility. Once the required predecessor version becomes visible, the background worker integrates the newly filled differential bitmap with the latest visible version and marks the placeholder available. If a query reaches a committed placeholder whose payload is not yet available, it waits until the filling and visibility stages complete, rather than skipping it and incorrectly returning an older version.

B. Basic Operations

We describe the insert, update, delete, and query operations under multi-threaded execution.



(a) Placeholder insertion (b) Differential filling (c) Visibility integration

Fig. 7: Concurrent bitmap insertion process of HierDiff

Algorithm 1: Delete Protocol

Input: New version v_{new} with commit sequence number CSN_{new}
Output: Updated version chain with v_{new} inserted
Phase 1: Placeholder Insertion;
if *current Reference group has remaining capacity* **then**
  Insert an empty placeholder node p into the group's version chain;
else
  Create a new Reference node R_{new} with all-zero bitmap;
  Link R_{new} to the front of the Reference chain;
  Insert placeholder node p under R_{new};
Record $CSN(p) \leftarrow CSN_{new}$;
Mark p as `unfilled`;
Phase 2: Differential Filling;
Construct differential bitmap Δ_{new};
Store Δ_{new} in sparse format within p;
Mark p as `filled`;
Phase 3: Visibility Integration;
if *all preceding versions in the group are visible* **then**
  Retrieve last visible version v_{prev};
  Integrate: $v_{new}.bitmap \leftarrow v_{prev}.bitmap \vee \Delta_{new}$;
  Update Reference node's CSN range: $[CSN_{min}, CSN_{max}] \leftarrow [CSN_{min}, CSN_{new}]$;
  Mark v_{new} as `visible`;
return updated version chain;

Insert. During the insertion procedure, incoming records are batched and compressed into a CU. Upon the creation of a CU, a corresponding two-level chain is initialized. The first bitmap version in the chain is created with all bits set to 0.

Delete. When a data record is deleted within a CU, HierDiff does not materialize the new bitmap version on the foreground path. Instead, the affected `Cu_id` and `Row_ids` are first recorded in the `Delete Log`. After the transaction enters the commit path and obtains its commit sequence number CSN_{new}, an empty placeholder is inserted into the two-level version chain at the correct logical position. Algorithm 1 illustrates the delete protocol, which consists of three phases corresponding to the *Placeholder–Filling–Visibility* mechanism. In the actual system, *Phase 1* is executed on the commit path, whereas *Phase 2* and *Phase 3* are completed asynchronously by a background worker.

Phase 1: Placeholder Insertion. On the commit path, HierDiff first determines whether the current `Reference` group, identified by its `CSN` range, has remaining capacity to accommodate a new version. If capacity is available, an empty placeholder node is inserted directly into the group's version chain. Otherwise, a new `Reference` version with an all-zero bitmap is created and linked to the head of the `Reference` chain, under which the placeholder node is inserted. The placeholder node is then assigned the commit sequence number CSN_{new} and marked as `unfilled`, reserving its position in the version order before the transaction returns.

Phase 2: Differential Filling. Next, a background worker scans the `Delete Log` in ascending `CSN` order, groups the corresponding records by `Cu_id` and `CSN`, and constructs a differential bitmap Δ_{new} for the delete operation by setting the bits of the affected `Row_ids` to 1. The differential bitmap is stored in sparse format within the placeholder node, after which the node is marked as `filled`. This phase performs bitmap construction without affecting version visibility.

Phase 3: Visibility Integration. Finally, once all preceding versions within the same `Reference` group become visible, HierDiff operates on the new version and makes it visible. Specifically, it retrieves the last visible bitmap version and modifies the new version by applying a bitwise OR operation with Δ_{new}. The `CSN` range of the corresponding `Reference` version is then updated to include CSN_{new}, and the new bitmap version is marked as `visible`.

Example. We illustrate the delete procedure in Fig. 7, where a new bitmap version with `CSN` $= 39$ is inserted. First, after the transaction obtains `CSN` $= 39$ on the commit path, an empty placeholder is inserted into the version chain under the first `Reference` version. At this point, the bitmap version with `CSN` $= 27$ is already visible in that `Reference` group. However, the immediate predecessor of the new version, i.e., the bitmap version with `CSN` $= 32$, has not yet become visible. Therefore, the `CSN` range of the `Reference` version remains $[27, 27]$. Second, the placeholder of `CSN` $= 39$ is filled using the corresponding `Delete Log` information, even though the `CSN` $= 32$ version is still pending visibility. Later, once `CSN` $= 32$ becomes visible, the integration process is triggered. Third, HierDiff integrates the latest visible version with the newly filled differential bitmap via a bitwise OR

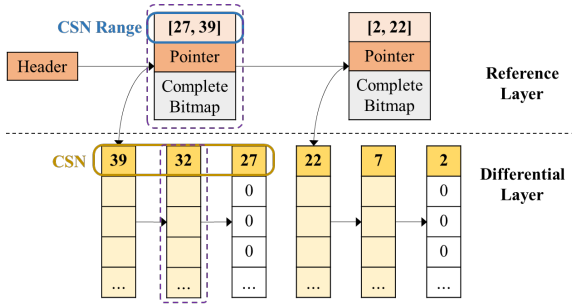


Fig. 8: Example: retrieving the version with $CSN = 35$

operation and updates the CSN range of the first Reference to $[27, 39]$, making the new version visible. This protocol ensures ordered and consistent version insertion, while reducing contention overhead and performance fluctuations under concurrent workloads.

Update. HierDiff adopts an append-only update method. When updating data in a CU, it logically deletes it by modifying the corresponding bitmap bits first, and then inserts the updated data into a new CU using the aforementioned methods.

Query. Fig. 8 illustrates an example of the search process. First, HierDiff locates the corresponding Reference version by searching the CSN range index in the first-level Reference layer chain. Next, within the associated differential layer, it identifies the target differential bitmap node whose CSN is less than or equal to the query CSN. Finally, the target version is reconstructed through a single XOR operation between the Reference bitmap and the selected differential bitmap. If the selected node is a committed placeholder whose filling and visibility stages have not yet completed, the query waits until the node becomes available, ensuring that committed updates are not skipped and older versions are not returned incorrectly.

Example. In the example illustrated in in Fig. 8, suppose we query the chain for CSN $= 35$. In the Reference layer, the CSN range of the first Reference version covers CSN $= 35$, so the query continues within its corresponding differential chain. In the differential layer, the nearest version whose CSN is less than or equal to 35 is the one with CSN $= 32$. The final bitmap for CSN $= 35$ is obtained by performing an XOR between this differential version and the complete bitmap maintained in the Reference version.

C. Garbage Collection

Algorithm 2 presents HierDiff's two-phase GC process: visibility-based reclamation and adaptive group merging.

Phase 1: Visibility-Based Reclamation. The system traverses the version chain and checks each version v_j under each Reference node R_i against the active transaction table T_{active}. Versions invisible to all active transactions are removed, and their differential bitmaps Δ_{v_j} and associated memory are released. After reclamation, the version count and CSN range of each R_i are updated.

Phase 2: Adaptive Group Merging. After reclamation, each group R_i whose version count falls below $\mathcal{T}$ is merged into its adjacent group R_{next}. The system computes the reference difference $bitmap_{merged} = R_i.bitmap_{ref} \oplus R_{next}.bitmap_{ref}$ and adjusts each remaining version under R_i using $v_k.bitmap \leftarrow v_k.bitmap \vee bitmap_{merged}$. The adjusted versions are appended to R_{next}'s group, its CSN range is updated to cover both groups, and the redundant Reference node R_i is removed.

Together, these phases reclaim obsolete versions and consolidate sparse groups, reducing memory overhead while maintaining efficient access.

Algorithm 2: Garbage Collection and Group Merging

Input: Active transaction table T_{active}, version chain C
Output: Compacted version chain with obsolete versions reclaimed

Phase 1: Visibility-based Reclamation;
foreach *Reference node* R_i *in chain* C **do**
  foreach *version* v_j *under* R_i **do**
    if v_j *is invisible to all transactions in* T_{active} **then**
      Remove v_j from version chain;
      Release bitmap Δ_{v_j} and related memory;
  Update R_i's version count and CSN range;
Phase 2: Adaptive Group Merging;
foreach *Reference node* R_i **do**
  if $R_i.version_count < \mathcal{T}$ **then**
    $bitmap_{merged} \leftarrow R_i.bitmap_{ref} \oplus R_{next}.bitmap_{ref}$;
    foreach *version* v_k *under* R_i **do**
      $v_k.bitmap \leftarrow v_k.bitmap \vee bitmap_{merged}$;
    Append updated versions to R_{next}'s group;
    Update $R_{next}.CSN_range$ and remove R_i;
return compacted version chain C;

VI. Experimental Evaluation

To assess the memory efficiency and performance impact of HierDiff, we integrate it into HexaDB [20] and perform comprehensive experiments addressing the following questions:

(Q1) How does HierDiff's memory efficiency compare to the chain-based multiversion scheme in the original HexaDB and other modern techniques?

(Q2) How precisely can HierDiff reclaim obsolete bitmap versions during garbage collection, compared with HexaDB and the interval-based GC baseline?

(Q3) What is the impact of HierDiff on system throughput and latency compared with HexaDB and other competing baselines?

(Q4) How does group size m affect memory efficiency, and do empirical results align with the theoretical optimum m^*?

(Q5) How does memory efficiency vary with the number of terminals?

(Q6) How does the merging threshold $\mathcal{T}$ affect memory efficiency, and do empirical results validate the theoretical optimum $\mathcal{T}^*$?

A. Experiment Setup

Implementation and Platform. We implement the original HexaDB and its augmented variants in C/C++. The source codes are available at https://github.com/qianluwu/HierDiff. All experiments are conducted on a CentOS 7.6 server equipped with an Intel Broadwell CPU (2.3 GHz, 64 cores). To evaluate performance under realistic memory constraints, the system is configured to operate with an effective memory budget of 4 GB.

Workloads. We evaluate our approach using two widely adopted benchmarks, TPC-C [30] and TPC-H [31], which represent typical transactional and analytical workloads, respectively. For both benchmarks, the data size is set to 100 GB. To simulate an HTAP scenario, we adapt the execution of TPC-H to generate a mixed transactional-analytical workload.

- **TPC-C**: This TP benchmark simulates a wholesale supplier scenario with a blend of update-heavy and read-only transactions. We use the BenchmarkSQL framework to execute the TPC-C workload, with 100 terminals running concurrently.
- **TPC-H**: This AP benchmark comprises 22 complex analytical queries over a read-only dataset. We augment it with updates and deletes for HTAP simulation. Specifically, we use 20 terminals to execute analytical queries, and 40 terminals to execute transactional operations, including 20 terminals for updates and 20 for deletes.

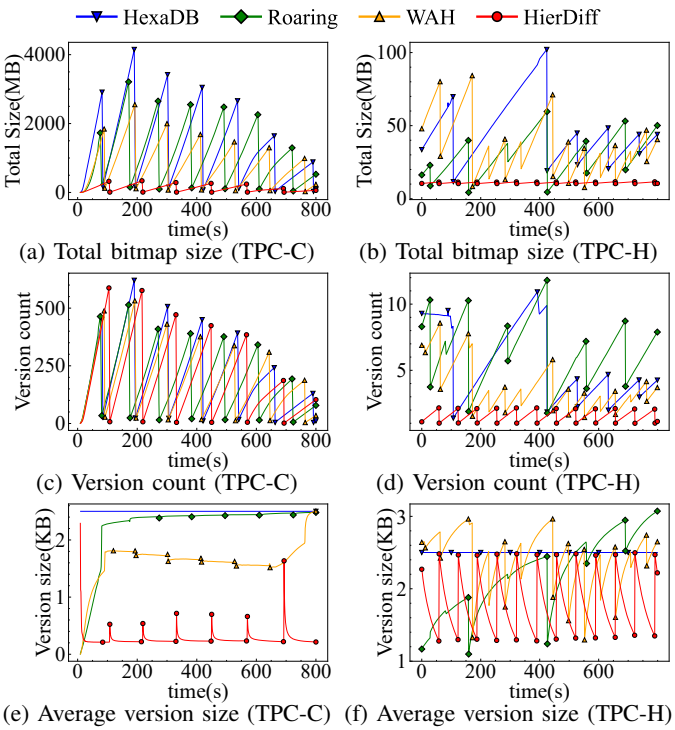
Unless otherwise specified, the above terminal configurations are used throughout all experiments.

Approaches and Metrics. We compare our HierDiff with several state-of-the-art techniques for bitmap compression and garbage collection. The evaluated approaches are as follows:

- HexaDB: the original HexaDB [20], which adopts a chain-based multiversion scheme and a garbage collection mechanism based on the classic *oldest active transaction* policy.
- Roaring: HexaDB with each bitmap version encoded using a Roaring-style compressed representation [29]. The bucket size is fixed at 2^{12} rows.
- WAH: HexaDB with each bitmap version compressed using the WAH encoding scheme [28].
- Interval: HexaDB integrated with an interval-based garbage collection scheme [12]. Active snapshot CSNs are grouped into intervals, and the interval size is set to 128 by default.
- HierDiff: HexaDB with the full HierDiff framework.

By default, HierDiff is configured with a group size $m = 25$ and a merging threshold $\mathcal{T} = 13$. These are derived from the theoretical analysis in Sec. IV and have been empirically validated as optimal for the TPC-C workload by the experiments reported in Sec. VI-E. When varying the group size, the merging threshold $\mathcal{T}$ is set to $\lceil m/2 \rceil$ by default.

The following metrics are used to evaluate system performance: (i) bitmap version storage overhead, including the total bitmap size, the bitmap version count, and the average version size; and (ii) transaction throughput and latency.



(a) Total bitmap size (TPC-C) (b) Total bitmap size (TPC-H)
(c) Version count (TPC-C) (d) Version count (TPC-H)
(e) Average version size (TPC-C) (f) Average version size (TPC-H)

Fig. 9: Bitmap Memory overhead comparison.

B. Memory Consumption (Q1)

To quantify the memory benefits of HierDiff, we measure three key metrics of the bitmap-version subsystem under TPC-C and TPC-H workloads: (i) *total bitmap size*, (ii) *average version count* of a single version chain, and (iii) *average version size*. For TPC-H, statistics are reported over a later 800-second window rather than the initial 800 seconds. This is because, in the early stage of execution, the bitmap versions remain relatively sparse and the number of set bits has not expanded sufficiently, which would overstate the compression effectiveness of Roaring and WAH. The later window provides a more representative view of steady-state memory behavior after sustained update and delete activity.

Total bitmap size. Figs. 9a and 9b show the evolution of total bitmap size over time under TPC-C and TPC-H, respectively. The sawtooth patterns reflect periodic memory reclamation by GC. HierDiff consistently achieves the smallest bitmap footprint among all schemes. Compared with HexaDB, HierDiff reduces total bitmap size by up to $12.3\times$ under TPC-C and $5.1\times$ under TPC-H. Notably, the bitmap size of TPC-C workloads is significantly larger than TPC-H due to its higher rate of inserts, deletes, and updates that create new versions. Although Roaring and WAH also reduce the footprint relative to the original HexaDB, their sizes remain substantially larger than HierDiff (i.e., by up to $9.5\times$ and $3.9\times$ versus Roaring, and $7.5\times$ and $4.1\times$ versus WAH under TPC-C and TPC-H). This demonstrates that hierarchical differential encoding controls memory growth more effectively than compact bitmap representations alone under sustained update pressure.

Version count. Figs. 9c and 9d illustrate the average version count per bitmap version chain. Under TPC-C, Hi-

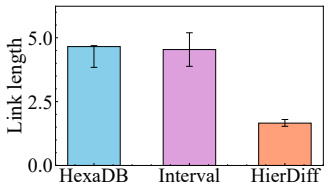


Fig. 10: Post-GC average link length

erDiff achieves a modest reduction relative to HexaDB. The improvement is more significant under TPC-H, where version count drops by up to $4.8\times$, and $2.2\times$ compared to the suboptimal WAH. The reason is that long-running analytical queries in TPC-H stall reclamation and allow obsolete versions to accumulate. HierDiff mitigates this through visibility-oriented GC with adaptive inter-group merging, enabling more aggressive and timely removal of obsolete bitmap versions.

Average Version Size. Figs. 9e and 9f show the average bitmap version sizes. Compared to HexaDB, HierDiff reduces average version size by 1.5-11.8$\times$ under TPC-C and 1.0-2.1$\times$ under TPC-H. As HexaDB stores uncompressed full bitmaps, its average version size remains constant throughout execution. For HierDiff, the average version size varies dynamically. Garbage collection reduces the number of differential versions, increasing average version size. Conversely, as new bitmap versions are inserted, the average version size gradually decreases. The greater reduction under TPC-C reflects longer version chains, which amplify the compression benefits of the hierarchical grouped differential encoding scheme. Roaring and WAH may initially achieve smaller average versions than HexaDB, but their compression ratio degrades as more bitmap bits become set. Under TPC-C, HierDiff's average version size is $10.6\times$ and $8.1\times$ lower than Roaring and WAH, respectively. These results show that HierDiff delivers robust memory savings across diverse workloads by combining differential encoding with aggressive reclamation.

C. Garbage Collection Precision (Q2)

We next evaluate the reclamation precision of HierDiff. Since the three schemes differ mainly in how precisely they retain versions that may still be visible to active transactions, we compare HexaDB, Interval, and HierDiff under the TPC-C workload. As the evaluation metric, we use the *post-GC average link length*, i.e., the average number of bitmap versions that remain on each CU chain immediately after a major garbage collection cycle. A lower value indicates fewer obsolete versions survive reclamation and hence higher GC policy precision.

Fig. 10 shows that HierDiff achieves the lowest post-GC average link length among the three schemes. After garbage collection, the average link length of HexaDB and Interval remains around 4.5-4.7 versions per chain, whereas HierDiff reduces it to about 1.7. This means that after each major GC, a chain under HierDiff typically retains only one to two versions, while the baseline schemes still keep about four to five versions on average.

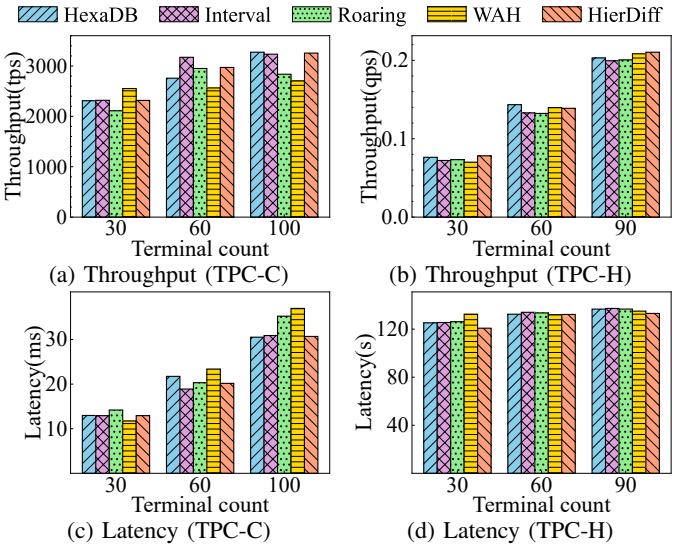


Fig. 11: Throughput and latency under TPC-C and TPC-H.

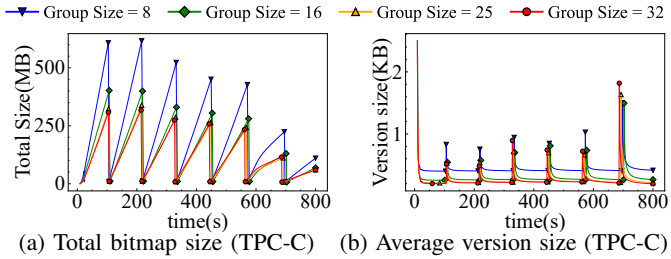
This result aligns with the design rationale in Sec. IV. HexaDB relies on a single global visibility frontier and conservatively keeps all newer versions. Interval refines this policy by grouping snapshot CSNs into fixed-size intervals, but all versions within partially active intervals must still be preserved conservatively. In contrast, HierDiff explicitly identifies versions that are actually visible to active snapshots and reclaims invisible intermediate versions more precisely, resulting in significantly shorter residual chains and lower future memory overhead.

D. System Efficiency (Q3)

To assess whether HierDiff achieves competitive runtime performance, we measure system throughput and latency under both TPC-C and TPC-H workloads. For TPC-H, 30, 60, and 90 terminals correspond to 10, 20, and 30 threads for each of the three operation types, and the results are measured on the AP operations.

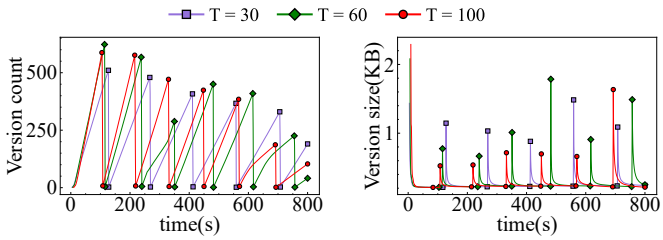
Throughput. Figs. 11a and 11b compare the throughput of the evaluated approaches as the number of terminals increases. HierDiff consistently matches or slightly exceeds the throughput of the original HexaDB. This indicates that the hierarchical differential encoding and deferred bitmap construction impose negligible oveerhead on the critical transaction path. Compared with the other alternatives, HierDiff also shows stable and competitive performance. At 100 terminals under TPC-C, its throughput reaches about $1.2\times$ that of WAH and $1.15\times$ that of Roaring. This suggests that HierDiff incurs lower bitmap encoding and decoding overhead under high concurrency. Interval performs competitively as it does not perform bitmap compression or decompression. Overall, HierDiff maintains high throughput while providing substantially stronger memory efficiency than the competing schemes.

Latency. Figs. 11c and 11d compare the transaction latency across all approaches. HierDiff exhibits latency comparable to that of the original HexaDB. This improvement mainly stems from the immediate commit protocol, which decouples bitmap



(a) Total bitmap size (TPC-C) (b) Average version size (TPC-C)

Fig. 12: Different group sizes under TPC-C.



(a) Version count (TPC-C) (b) Average version size (TPC-C)

Fig. 13: Different #terminals under TPC-C.

materialization from the transaction path and effectively reduces transaction waiting time. As concurrency increases, the latency raises for all schemes, yet HierDiff remains among the best performers. At 100 terminals under TPC-C, it achieves approximately 17% lower latency than WAH and 13% lower latency than Roaring. These results confirm that the memory optimizations of HierDiff do not come at the expense of significant runtime overhead.

E. Memory Cost vs. Group Size (Q4)

In Sec. IV-A, we analyze the selection of the group size m and derive the optimal value m^*. Observing the 99.98% similarity between adjacent bitmap versions, we set $p = 0.02\%$ (i.e., $t = 0.9998$). With $n = 400$, the analytical model yields $m^* \approx 25.5$, which we round to 25 in practice, corresponding to a predicted total bitmap size of approximately 6.15×10^5 bits per two-level chain. We experimentally evaluate different group sizes to validate this theoretical optimum.

Since group size does not affect GC behaviors, we focus on total bitmap size and average version size. As shown in Fig. 12, the configuration with $m = 25$ consistently achieves the lowest total bitmap size and average version size, confirming the accuracy of the analytical model. When m exceeds 25 (e.g., $m = 32$), the total bitmap size remains close to that of $m = 25$. This observation is consistent with the theoretical analysis in Sec. IV-A: when m is small, increasing m significantly reduces memory overhead, whereas once m exceeds the optimal region, the memory savings diminish and the total size becomes relatively stable. Overall, these results confirm that choosing a group size close to the theoretically optimal value achieves near-optimal memory efficiency in practice.

F. Memory Cost vs. Terminal Number (Q5)

To evaluate the impact of terminal concurrency on memory overhead, we vary the number of terminals and measure bitmap version count and average version size under TPC-C workloads. Results are presented in Fig. 13. As the number of terminals increases, the bitmap version count consistently grows across both workloads. This is because higher concurrency generates more transactions, leading to the two-level version chain to grow longer. After each garbage collection cycle, the version count drops to a similar level, then rises again at different rates w.r.t. different terminal configurations.

Regarding the average version size, we observe a decreasing trend as the number of terminals increases. This is because a longer version chain enables the hierarchical grouped differential encoding scheme to exploit similarity across more adjacent versions, thereby achieving higher compression efficiency and reducing the average size of each bitmap version.

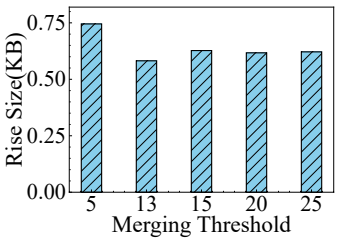


Fig. 14: Merging threshold comparison

G. Memory Cost vs. Merging Threshold (Q6)

In Sec. IV-B, we derive an analytical expression for the optimal merging threshold v. In this experiment, we continue to use $n = 400$, $m^* = 25$, and $t = 0.9998$. Since the number of groups is approximately $n/m^* \approx 16$, we substitute these values into the analytical model and examine how v varies with the group index z along the version chain. We observe that v remains relatively stable across different groups. Specifically, for z ranging from 1 to 16, the computed values of v fall within a narrow range of 12 to 14. For simplicity, we select $v = 13$ as the merging threshold and denote it as $\mathcal{T}$ in the following experiments. We then vary the merging threshold $\mathcal{T}$ around this value and measure the resulting memory overhead.

During execution, each garbage collection cycle leads to an increase in the average version size, since obsolete versions in the differential layer are reclaimed, reducing the total version count. However, when inter-group merging is triggered, the magnitude of this increase becomes smaller. This is because group merging moves differential bitmap versions from one group into its adjacent group, while the complete bitmap stored in the `Reference` layer of the merged group is removed. As a result, although the version count decreases, the total bitmap size is reduced more effectively, leading to a smaller rebound in the average version size. A smaller increase therefore indicates a more effective merging operation.

As shown in Fig. 14, among all tested merging threshold settings, $\mathcal{T} = 13$ results in the smallest increase in average version size after garbage collection and group merging. This result validates the effectiveness of our analytical model on merging threshold.

VII. Related Work

A. MVCC in Column Stores

MVCC designs for column-store systems fall broadly into two families based on how tuple visibility is represented: (i) systems embedding persistent visibility bitmaps in immutable column segments, and (ii) systems storing MVCC metadata in version chains or undo logs while using bitmaps only as temporary execution-time filters.

In the first category, persistent bitmap-based MVCC embeds visibility information directly into the columnar format. For instance, SAP HANA [12], [32] utilizes immutable main fragments augmented with deletion bitmaps to denote logically invalid tuples, with inserts and updates buffered in delta fragments. Snapshot visibility is thus determined by combining per-segment deletion bitmaps with commit-time metadata. Similarly, openGauss [10] incorporates delete bitmaps per Compression Unit and augments them with PostgreSQL-style `xmin/xmax` metadata. Although these approaches enable efficient vectorized filtering, they typically store each version in full without cross-version compression, incurring considerable memory overhead from bitmap histories.

The second category relies on version-chain or undo-log, where bitmaps serve only as temporary selection masks during vectorized scans. Hyper [11] employs per-tuple version chains with begin-end timestamps, resolving visibility via these chains and generating temporary selection bitmaps at runtime. DuckDB [33] appends updates to immutable column segments while recording prior states in undo buffers. ByteHTAP [34] employs undo-style versioning across its primary column store and row-based delta store. Since MVCC metadata here is decoupled from persistent bitmaps, these systems avoid bitmap expansion but offer no mechanism for compressing or reorganizing bitmap histories.

B. Garbage Collection in MVCC

Garbage collection in MVCC systems reclaims obsolete versions that are no longer visible to any active transaction. Most systems determine reclaimable versions by tracking a global visibility frontier (e.g., the *oldest active transaction* or snapshot) and removing versions whose commit timestamps precede this boundary.

In bitmap-centric column stores such as SAP HANA [32] and openGauss [10], GC is a heavyweight process tightly coupled with physical data reorganization. SAP HANA merges delta fragments and rewrites main segments based on deletion bitmaps. Similarly, openGauss rewrites entire Compression Units when they contain a high ratio of deleted tuples. Because bitmap-guided reconstruction involves decompressing and rewriting columnar data, such heavyweight GC operations are triggered infrequently.

In contrast, systems based on version chains or undo logs, such as Hyper [11], DuckDB [33], and ByteHTAP [34], reclaim space by trimming expired version metadata once versions become invisible to all transactions. As MVCC metadata is stored separately from immutable column segments, GC affects only version chains or undo buffers without rewriting columnar storage. Bitmaps are transient execution-time structures that play no role in reclamation.

In summary, a clear gap exists in the literature. Bitmap-based MVCC systems suffer from severe memory overhead due to a lack of cross-version compression and reliance on costly GC. Our work targets this gap by proposing a lightweight, fine-grained mechanism to manage and compress bitmap version histories, improving memory efficiency without sacrificing performance.

VIII. Conclusion

In this paper, we present a novel framework, HierDiff, for optimized multiversion management in HTAP databases. Through an innovative hierarchical grouped differential encoding scheme and an adaptive garbage collection mechanism, HierDiff achieves optimized memory utilization. The proposed immediate commit protocol and two-level version chain further contribute to maintain competitive system performance across both OLAP and OLTP workloads. Extensive experiments validate its effectiveness and efficiency.

REFERENCES

[1] U. Sirin and A. Ailamaki, "Micro-architectural analysis of OLAP: limitations and opportunities," *Proc. VLDB Endow.*, vol. 13, no. 6, pp. 840–853, 2020.

[2] H. Plattner, "A common database approach for oltp and olap using an in-memory column database," *Proceedings of the VLDB Endowment*, vol. 1, no. 2, pp. 706–717, 2013.

[3] M. J. Cahill, U. Röhm, and A. D. Fekete, "Serializable isolation for snapshot databases," *ACM Trans. Database Syst.*, vol. 34, no. 4, pp. 20:1–20:42, 2009.

[4] U. M. Widenius and D. Axmark, *MySQL reference manual - documentation from the source*. O'Reilly, 2002.

[5] P. A. Bernstein, V. Hadzilacos, and N. Goodman, *Concurrency Control and Recovery in Database Systems*. Addison-Wesley, 1987.

[6] J. Gray and A. Reuter, *Transaction Processing: Concepts and Techniques*. Morgan Kaufmann, 1993.

[7] J. Kim, J. Yu, J. Ahn, S. Kang, and H. Jung, "Diva: Making MVCC systems htap-friendly," in *SIGMOD Conference*. ACM, 2022, pp. 49–64.

[8] T. Lahiri, S. Chavan, M. Colgan, D. Das, A. Ganesh, M. Gleeson, S. Hase, A. Holloway, J. Kamp, T. Lee, J. Loaiza, N. MacNaughton, V. Marwah, N. Mukherjee, A. Mullick, S. Muthulingam, V. Raja, M. Roth, E. Soylemez, and M. Zaït, "Oracle database in-memory: A dual format in-memory database," in *ICDE*. IEEE Computer Society, 2015, pp. 1253–1258.

[9] F. Färber, S. K. Cha, J. Primsch, C. Bornhövd, S. Sigg, and W. Lehner, "SAP HANA database: data management for modern business applications," *SIGMOD Rec.*, vol. 40, no. 4, pp. 45–51, 2011.

[10] G. Li, X. Zhou, J. Sun, X. Yu, Y. Han, L. Jin, W. Li, T. Wang, and S. Li, "opengauss: An autonomous database system," *Proc. VLDB Endow.*, vol. 14, no. 12, pp. 3028–3041, 2021.

[11] A. Kemper and T. Neumann, "Hyper: A hybrid oltp&olap main memory database system based on virtual memory snapshots," in *ICDE*. IEEE Computer Society, 2011, pp. 195–206.

[12] J. Lee, H. Shin, C. G. Park, S. Ko, J. Noh, Y. Chuh, W. Stephan, and W. Han, "Hybrid garbage collection for multi-version concurrency control in SAP HANA," in *SIGMOD Conference*. ACM, 2016, pp. 1307–1318.

[13] K. Kim, T. Wang, R. Johnson, and I. Pandis, "ERMIA: fast memory-optimized database system for heterogeneous workloads," in *SIGMOD Conference*. ACM, 2016, pp. 1675–1687.

[14] PostgreSQL Global Development Group, "Postgresql," https://www.postgresql.org, 2025, retrieved December 1, 2025.

[15] Oracle, "Mysql," https://www.mysql.com, 2025, retrieved December 1, 2025.

[16] M. Stonebraker, D. J. Abadi, A. Batkin, X. Chen, M. Cherniack, M. Ferreira, E. Lau, A. Lin, S. Madden, E. J. O'Neil, P. E. O'Neil, A. Rasin, N. Tran, and S. B. Zdonik, "C-store: A column-oriented DBMS," in *VLDB*. ACM, 2005, pp. 553–564.

[17] S. Idreos, F. Groffen, N. Nes, S. Manegold, K. S. Mullender, and M. L. Kersten, "Monetdb: Two decades of research in column-oriented database architectures," *IEEE Data Eng. Bull.*, vol. 35, no. 1, pp. 40–45, 2012.

[18] F. Färber, N. May, W. Lehner, P. Große, I. Müller, H. Rauhe, and J. Dees, "The SAP HANA database – an architecture overview," *IEEE Data Eng. Bull.*, vol. 35, no. 1, pp. 28–33, 2012.

[19] C. Zhang, G. Li, J. Zhang, X. Zhang, and J. Feng, "HTAP databases: A survey," *IEEE Trans. Knowl. Data Eng.*, vol. 36, no. 11, pp. 6410–6429, 2024.

[20] Shuyi Technology, "Hexadb," https://www.hexadb.com, 2025, retrieved December 27, 2025.

[21] H. Berenson, P. A. Bernstein, J. Gray, J. Melton, E. J. O'Neil, and P. E. O'Neil, "A critique of ANSI SQL isolation levels," in *SIGMOD Conference*. ACM Press, 1995, pp. 1–10.

[22] A. D. Fekete, D. Liarokapis, E. J. O'Neil, P. E. O'Neil, and D. E. Shasha, "Making snapshot isolation serializable," *ACM Trans. Database Syst.*, vol. 30, no. 2, pp. 492–528, 2005.

[23] P. Larson, S. Blanas, C. Diaconu, C. Freedman, J. M. Patel, and M. Zwilling, "High-performance concurrency control mechanisms for main-memory databases," *Proc. VLDB Endow.*, vol. 5, no. 4, pp. 298–309, 2011.

[24] J. Shute, R. Vingralek, B. Samwel, B. Handy, C. Whipkey, E. Rollins, M. Oancea, K. Littlefield, D. Menestrina, S. Ellner, J. Cieslewicz, I. Rae, T. Stancescu, and H. Apte, "F1: A distributed SQL database that scales," *Proc. VLDB Endow.*, vol. 6, no. 11, pp. 1068–1079, 2013.

[25] C. Riegger, T. Vinçon, R. Gottstein, and I. Petrov, "MV-PBT: multi-version indexing for large datasets and HTAP workloads," in *EDBT*. OpenProceedings.org, 2020, pp. 217–228.

[26] A. Lamb, M. Fuller, R. Varadarajan, N. Tran, B. Vandiver, L. Doshi, and C. Bear, "The vertica analytic database: C-store 7 years later," *Proc. VLDB Endow.*, vol. 5, no. 12, pp. 1790–1801, 2012.

[27] H. Song, W. Zhou, H. Cui, X. Peng, and F. Li, "A survey on hybrid transactional and analytical processing," *VLDB J.*, vol. 33, no. 5, pp. 1485–1515, 2024.

[28] K. Wu, E. J. Otoo, and A. Shoshani, "Optimizing bitmap indices with efficient compression," *ACM Trans. Database Syst.*, vol. 31, no. 1, pp. 1–38, 2006.

[29] S. Chambi, D. Lemire, O. Kaser, and R. Godin, "Better bitmap performance with roaring bitmaps," *Softw. Pract. Exp.*, vol. 46, no. 5, pp. 709–719, 2016.

[30] Transaction Processing Performance Council, "Tpc-c benchmark," https://www.tpc.org/tpcc/, 2025, retrieved December 31, 2025.

[31] ——, "Tpc-h benchmark," https://www.tpc.org/tpch/, 2025, retrieved December 31, 2025.

[32] V. Sikka, F. Färber, W. Lehner, S. K. Cha, T. Peh, and C. Bornhövd, "Efficient transaction processing in SAP HANA database: the end of a column store myth," in *SIGMOD Conference*. ACM, 2012, pp. 731–742.

[33] M. Raasveldt and H. Mühleisen, "Duckdb: an embeddable analytical database," in *SIGMOD Conference*. ACM, 2019, pp. 1981–1984.

[34] J. Chen, Y. Ding, Y. Liu, F. Li, L. Zhang, M. Zhang, K. Wei, L. Cao, D. Zou, Y. Liu, L. Zhang, R. Shi, W. Ding, K. Wu, S. Luo, J. Sun, and Y. Liang, "Bytehtap: Bytedance's HTAP system with high data freshness and strong data consistency," *Proc. VLDB Endow.*, vol. 15, no. 12, pp. 3411–3424, 2022.